Reconstruction

26th Sep 2024 / Document No. D24.102.258

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: Very Easy

Classification: Official

Synopsis

Reconstruction is a very easy difficulty challenge that features writing `assembly` to change the values of some registers.

Description

One of the Council's divine weapons has its components, known as registers, misaligned. Can you restore them and revive this ancient weapon?

Skills Required

- Basic C, `assembly`.

Skills Learned

- Crafting custom payload in `assembly` to change the values of registers.

Enumeration

First of all, we start with a `checksec`:

```
pwndbg> checksec
Arch:     amd64
RELRO:      Full RELRO
Stack:      Canary found
NX:         NX unknown - GNU_STACK missing
PIE:        PIE enabled
Stack:      Executable
RWX:        Has RWX segments
RUNPATH:    b'./glibc/'
SHSTK:      Enabled
IBT:        Enabled
Stripped:   No
```

Protections 🛡️

As we can see:

Protection	Enabled	Usage
Canary	✅	Prevents **Buffer Overflows**
NX	❌	Disables **code execution** on stack
PIE	✅	Randomizes the **base address** of the binary
RelRO	**Full**	Makes some binary sections **read-only**

The program's interface:

```
[*] Initializing components...

[-] Error: Misaligned components!

[*] If you intend to fix them, type "fix": fix

[!] Carefully place all the components: w3t

[-] Invalid byte detected: 0x77 at position 0

[-] Invalid payload! Execution denied.
```

As we can see, there is some kind of "byte" checking. We cannot understand more things from here, so let's open our decompiler.

Disassembly

Starting with `main()`:

```
00001b28  int32_t main(int32_t argc, char** argv, char** envp)

00001b28  {
00001b3d      void* fsbase;
00001b3d      int64_t var_10 = *(uint64_t*)((char*)fsbase + 0x28);
00001b43      banner();
00001b48      int32_t choice = 0;
00001b4f      char var_11 = 0;
00001b5d      printstr("\n[*] Initializing components...…");
00001b67      sleep(1);
00001b76      puts("\x1b[1;31m");
00001b85      printstr("[-] Error: Misaligned components…");
00001b94      puts("\x1b[1;34m");
00001ba3      printstr("[*] If you intend to fix them, t…");
00001bb9      read(0, &choice, 4);
00001bb9
00001bdb      if (strncmp(&choice, &data_344c, 3) != 0)
00001bdb      {
00001c1f          puts("\x1b[1;31m");
00001c2e          printstr("[-] Mission failed!\n\n");
00001c38          exit(0x520);
00001c38          /* no return */
00001bdb      }
00001bdb
00001be7      puts("\x1b[1;33m");
```

```
00001bf6      printstr("[!] Carefully place all the comp…");
00001bf6
00001c07      if (check() != 0)
00001c0e          read_flag();
00001c0e
00001c42      exit(0x520);
00001c42      /* no return */
00001b28  }
```

The goal here is straightforward:

We need to enter the word "fix" and the, if `check()` returns true, it prints the flag. Let's take a look at `check()`.

```
0000189c  int64_t check()

0000189c  {
000018a9      void* fsbase;
000018a9      int64_t canary = *(uint64_t*)((char*)fsbase + 0x28);
000018d8      int64_t* exec_mem = mmap(nullptr, 0x3c, 7, 0x22, 0xffffffff, 0);
000018d8
000018e6      if (exec_mem == -1)
000018e6      {
000018f2          perror("mmap");
000018fc          exit(1);
000018fc          /* no return */
000018e6      }
000018e6
00001901      int64_t payload;
00001901      __builtin_memset(&payload, 0, 0x3d);
00001952      read(0, &payload, 0x3c);
00001963      *(uint64_t*)exec_mem = payload;
00001966      __builtin_memset(&exec_mem[1], 0, 0x20);
00001986      int64_t var_40;
00001986      exec_mem[5] = var_40;
00001992      *(uint64_t*)((char*)exec_mem + 0x2d) = var_40;
00001996      int64_t var_33;
00001996      *(uint64_t*)((char*)exec_mem + 0x35) = var_33;
00001996
000019ad      if (validate_payload(exec_mem, 0x3b) == 0)
000019ad      {
000019b9          error("Invalid payload! Execution denie…");
000019c3          exit(1);
000019c3          /* no return */
000019ad      }
000019ad
000019d9      exec_mem();
000019e7      munmap(exec_mem, 0x3c);
000019ec      char counter = 0;
00001b03      int64_t result;
00001b03
00001b03      while (true)
00001b03      {
00001b03          if (counter > 6)
00001b03          {
```

```
00001b09              result = 1;
00001b09              break;
00001b03          }
00001b03
00001a33          int64_t r12;
00001a33          int64_t r13;
00001a33          int64_t r14;
00001a33          int64_t r15;
00001a33
00001a33          if (regs(&buf[((int64_t)((uint32_t)counter))], r12, r13, r14,
r15) != *(uint64_t*)((((int64_t)((uint32_t)counter)) << 3) + &values))
00001a33          {
00001a4e              int64_t rbx_2 = *(uint64_t*)((((int64_t)
((uint32_t)counter)) << 3) + &values);
00001a6e              int64_t rax_16 = regs(&buf[((int64_t)((uint32_t)counter))],
r12, r13, r14, r15);
00001ae5              printf("%s\n[-] Value of [ %s$%s%s ]: [ …", "\x1b[1;31m",
"\x1b[1;35m", &buf[((int64_t)((uint32_t)counter))], "\x1b[1;31m", "\x1b[1;35m",
rax_16, "\x1b[1;31m", "\x1b[1;32m", "\x1b[1;33m", rbx_2, "\x1b[1;32m");
00001aee              result = 0;
00001af3              break;
00001a33          }
00001a33
00001afc          counter += 1;
00001b03      }
00001b03
00001b12      *(uint64_t*)((char*)fsbase + 0x28);
00001b12
00001b1b      if (canary == *(uint64_t*)((char*)fsbase + 0x28))
00001b27          return result;
00001b27
00001b1d      __stack_chk_fail();
00001b1d      /* no return */
0000189c  }
```

As we can see, our payload is inserted in the `payload` buffer, it checks for `whitelisted` bytes, and then executes it.

```
00001952      read(0, &payload, 0x3c);
00001963      *(uint64_t*)exec_mem = payload;
00001966      __builtin_memset(&exec_mem[1], 0, 0x20);
00001986      int64_t var_40;
00001986      exec_mem[5] = var_40;
00001992      *(uint64_t*)((char*)exec_mem + 0x2d) = var_40;
00001996      int64_t var_33;
00001996      *(uint64_t*)((char*)exec_mem + 0x35) = var_33;
00001996
000019ad      if (validate_payload(exec_mem, 0x3b) == 0)
000019ad      {
000019b9          error("Invalid payload! Execution denie…");
000019c3          exit(1);
000019c3          /* no return */
000019ad      }
000019ad
000019d9      exec_mem();
```

```
000019e7      munmap(exec_mem, 0x3c);
```

The `validate_payload` function is this:

```
000013a9  int64_t validate_payload(int64_t arg1, int64_t arg2)

000013a9  {
000013bd      void* fsbase;
000013bd      int64_t rax = *(uint64_t*)((char*)fsbase + 0x28);
000013cc      void* var_20 = nullptr;
00001474      int64_t result;
00001474
00001474      while (true)
00001474      {
00001474          if (var_20 >= arg2)
00001474          {
0000147a              result = 1;
0000147a              break;
00001474          }
00001474
000013d9          int32_t var_24_1 = 0;
000013d9
00001420          for (int64_t i = 0; i <= 0x11; i += 1)
00001420          {
0000140b              if (*(uint8_t*)((char*)var_20 + arg1) == *(uint8_t*)(i +
&allowed_bytes))
0000140b              {
0000140d                  var_24_1 = 1;
00001414                  break;
0000140b              }
00001420          }
00001420
00001426          if (var_24_1 == 0)
00001426          {
0000145b              printf("%s\n[-] Invalid byte detected: 0…", "\x1b[1;31m",
((uint64_t)*(uint8_t*)((char*)var_20 + arg1)), var_20);
00001460              result = 0;
00001465              break;
00001426          }
00001426
00001467          var_20 += 1;
00001474      }
00001474
00001483      *(uint64_t*)((char*)fsbase + 0x28);
00001483
0000148c      if (rax == *(uint64_t*)((char*)fsbase + 0x28))
00001494          return result;
00001494
0000148e      __stack_chk_fail();
0000148e      /* no return */
000013a9  }
```

Taking a look at the `allowed_bytes` array:

```
const uint8_t data[32] =
{
    0x49, 0xc7, 0xb9, 0xc0, 0xde, 0x37, 0x13, 0xc4, 0xc6, 0xef, 0xbe, 0xad, 0xca,
0xfe, 0xc3, 0x00,
    0xba, 0xbd, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00
};
```

So, our payload must contain only these bytes to be accepted. After that, we can take a look how the program flow continues.

```
00001a33          if (regs(&buf[((int64_t)((uint32_t)counter))], r12, r13, r14,
r15) != *(uint64_t*)((((int64_t)((uint32_t)counter)) << 3) + &values))
00001a33          {
00001a4e              int64_t rbx_2 = *(uint64_t*)((((int64_t)
((uint32_t)counter)) << 3) + &values);
00001a6e              int64_t rax_16 = regs(&buf[((int64_t)((uint32_t)counter))],
r12, r13, r14, r15);
00001ae5              printf("%s\n[-] Value of [ %s$%s%s ]: [ …", "\x1b[1;31m",
"\x1b[1;35m", &buf[((int64_t)((uint32_t)counter))], "\x1b[1;31m", "\x1b[1;35m",
rax_16, "\x1b[1;31m", "\x1b[1;32m", "\x1b[1;33m", rbx_2, "\x1b[1;32m");
00001aee              result = 0;
00001af3              break;
00001a33          }
```

We see a call to `regs(char *s)` function.

```
00001495  int64_t regs(char* arg1, int64_t arg2 @ r12, int64_t arg3 @ r13,
int64_t arg4 @ r14, int64_t arg5 @ r15)

00001495  {
000014a5      void* fsbase;
000014a5      int64_t rax = *(uint64_t*)((char*)fsbase + 0x28);
000014b4      int64_t result = 0;
000014cd      int32_t rax_2;
000014cd      int64_t result_1;
000014cd      rax_2 = strcmp(arg1, &data_2008);
000014cd
000014d4      if (rax_2 != 0)
000014d4      {
000014f3          int32_t rax_5;
000014f3          int64_t result_2;
000014f3          rax_5 = strcmp(arg1, &data_200b);
000014f3
000014fa          if (rax_5 != 0)
000014fa          {
00001519              int32_t rax_8;
00001519              int64_t result_3;
00001519              rax_8 = strcmp(arg1, &data_200e);
00001519
00001520              if (rax_8 != 0)
00001520              {
00001546                  if (strcmp(arg1, &data_2012) != 0)
00001546                  {
```

```
0000156c                    if (strcmp(arg1, &data_2016) != 0)
0000156c                    {
0000158f                        if (strcmp(arg1, &data_201a) != 0)
0000158f                        {
000015b2                            if (strcmp(arg1, &data_201e) != 0)
000015d3                                printf("Unknown register: %s\n", arg1);
000015b2                            else
000015b7                                result = arg5;
0000158f                        }
0000158f                        else
00001594                            result = arg4;
0000156c                    }
0000156c                    else
00001571                        result = arg3;
00001546                }
00001546                else
0000154b                    result = arg2;
00001520            }
00001520            else
00001525                result = result_3;
000014fa        }
000014fa        else
000014ff            result = result_2;
000014d4    }
000014d4    else
000014d9        result = result_1;
000014d9
000015e0    *(uint64_t*)((char*)fsbase + 0x28);
000015e0
000015e9    if (rax == *(uint64_t*)((char*)fsbase + 0x28))
000015f1        return result;
000015f1
000015eb    __stack_chk_fail();
000015eb    /* no return */
00001495  }
```

If we analyze what these "data_" variables are, we see its an array containing the values:

```
r8, r9, r10, r12, r13, r14 and r15.
```

And it compares the values in these registers with the `values` buffer which contains:

```
const uint64_t data[8] =
{
    0x000000001337c0de, 0x00000000deadbeef, // r8, r9
    0x00000000dead1337, 0x0000000001337cafe, // r10, r12
    0x00000000beefc0de, 0x0000000013371337, // r13, r14
    0x000000001337dead                      // r15
};
```

So, our goal is to make the register above, contain the values we want. The assembly code to do that is:

```
sc = asm(f'''
    mov r8,  0x1337c0de
    mov r9,  0xdeadbeef
    mov r10, 0xdead1337
    mov r12, 0x1337cafe
    mov r13, 0xbeefc0de
    mov r14, 0x13371337
    mov r15, 0x1337dead
    ret
''')
```